

C++ Standard Library Issues to be moved in Kona

Doc. no. R0165R4

Date: Revised 2017-02-06 at 04:02:09 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[2260](#). Missing requirement for `Allocator::pointer`

Section: 17.5.3.5 [allocator.requirements] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2013-05-14 **Last modified:** 2017-02-03

Priority: 3

View other [active issues](#) in [allocator.requirements].

View all other [issues](#) in [allocator.requirements].

View all issues with [Tentatively Ready](#) status.

Discussion:

For an allocator `A<T>` which defines `A<T>::pointer` to a class type, i.e. not `T*`, I see no requirement that `A<T>::pointer` is convertible to `A<U>::pointer`, even if `T*` is convertible to `U*`. Such conversions are needed in containers to convert from e.g. `ListNodeBase*` to `ListNode<T>*`.

The obvious way to do such conversions appears to be `pointer_traits::pointer_to()`, but that's ill-formed if the static member function `A<T>::pointer::pointer_to()` doesn't exist and the allocator requirements don't mention that function, so you need to cast `A<T>::pointer` to `A<T>::void_pointer` then cast that to `A<U>::pointer`.

Is converting via `void_pointer` really intended, or are we missing a requirement that `pointer_traits<A<T>::pointer>::pointer_to()` be well-formed?

Proposed resolution:

Add to the Allocator requirements table the following requirement:

The expression `pointer_traits<XX::pointer>::pointer_to(r)` is well-defined.

[2013-09 Chicago]

Pablo to come back with proposed wording

[2015-07 Telecon]

Marshall to ping Pablo for proposed wording and disable current wording.

Previous resolution [SUPERSEDED]:

1. Edit Table 28 as indicated:

Table 28 — Allocator requirements (continued)

Expression	Return type	Assertion/note condition	pre-/post-	Default
...				
<code>static_cast<X::const_pointer>(z)</code>	<code>X::const_pointer</code>	<code>static_cast<X::const_pointer>(z) == q</code>		
<code>pointer_traits<X::pointer>::pointer_to(r)</code>	<code>X::pointer</code>			
...				

[2016-11-12, Issaquah]

This is related to [1521](#).

Sat PM: Restore original P/R and move to tentatively ready.

Proposed resolution:

- 1. Edit Table 28 as indicated:

Table 28 — Allocator requirements (continued)			
Expression	Return type	Assertion/note pre-/post-condition	Default
...			
static_cast<X::const_pointer>(z)	X::const_pointer	static_cast<X::const_pointer>(z) == q	
pointer_traits<X::pointer>::pointer_to(r)	X::pointer		
...			

2768. any_cast and move semantics

Section: 20.8.4 [any.nonmembers] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2016-08-27 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [any.nonmembers].

View all other [issues](#) in [any.nonmembers].

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG [2509](#) made two changes to the specification of any in v2 of the library fundamentals TS:

- 1. It altered the effects of the any_cast(any&&) overload to enable moving the value out of the any object and/or obtaining an rvalue reference to the contained value.
- 2. It made changes to support pathological copyable-but-not-movable contained values, which is madness.

Change 1 has very desirable effects; I propose that we apply the sane part of LWG [2509](#) to any in the C++17 WP, for all of the reasons cited in the discussion of LWG [2509](#).

[2016-09-09 Issues Resolution Telecon]

P0; move to Tentatively Ready

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

- 1. In 20.8.4 [any.nonmembers] p5, edit as follows:

```
template<class ValueType>
    ValueType any_cast(const any& operand);
template<class ValueType>
    ValueType any_cast(any& operand);
template<class ValueType>
    ValueType any_cast(any&& operand);
```

- 4- *Requires:* is_reference_v<ValueType> is true OR is_copy_constructible_v<ValueType> is true. Otherwise the program is ill-formed.
- 5- *Returns:* For the first form, *any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand). For the second and third forms, *any_cast<remove_reference_t<ValueType>>(&operand). For the third form, std::forward<ValueType>(*any_cast<remove_reference_t<ValueType>>(&operand)).

[Issues Telecon 16-Dec-2016]

Move to Tentatively Ready

Proposed resolution:

Resolved by the wording provided by LWG [2769](#).

[2769](#). Redundant `const` in the return type of `any_cast(const any&)`

Section: 20.8.4 [any.nonmembers] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2016-09-02 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [any.nonmembers].

View all other [issues](#) in [any.nonmembers].

View all issues with [Tentatively Ready](#) status.

Discussion:

The overload of `any_cast` that accepts a reference to constant `any`:

```
template<class ValueType>
ValueType any_cast(const any& operand);
```

is specified to return `*any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand)` in [any.nonmembers]/5. This calls the pointer-to-constant overload of `any_cast`:

```
template<class ValueType>
const ValueType* any_cast(const any* operand) noexcept;
```

which is specified as:

Returns: If `operand != nullptr` && `operand->type() == typeid(ValueType)`, a pointer to the object contained by `operand`; otherwise, `nullptr`.

Since `typeid(T) == typeid(const T)` for all types `T`, `any_cast<add_const_t<T>>(&operand)` is equivalent to `any_cast<T>(&operand)` for all types `T` when `operand` is a constant lvalue `any`. The `add_const_t` in the return specification of the first overload above is therefore redundant.

[2016-09-09 Issues Resolution Telecon]

P0; move to Tentatively Ready

Casey will provide combined wording for this and [2768](#), since they modify the same paragraph.

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Modify 20.8.4 [any.nonmembers] as indicated:

```
template<class ValueType>
ValueType any_cast(const any& operand);
template<class ValueType>
ValueType any_cast(any& operand);
template<class ValueType>
ValueType any_cast(any&& operand);
```

-4- *Requires:* `is_reference_v<ValueType>` is true OR `is_copy_constructible_v<ValueType>` is true. Otherwise the program is ill-formed.

-5- *Returns:* For the first form, `*any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand)`; For the second and third forms, `*any_cast<remove_reference_t<ValueType>>(&operand)`.

[...]

[2016-09-09 Casey improves wording as determined by telecon]

The presented resolution is intended as the common wording for both LWG [2768](#) and LWG [2769](#).

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Modify 20.8.4 [any.nonmembers] as indicated:

```
template<class ValueType>
    ValueType any_cast(const any& operand);
template<class ValueType>
    ValueType any_cast(any& operand);
template<class ValueType>
    ValueType any_cast(any&& operand);
```

-4- *Requires:* `is_reference_v<ValueType>` is true OR `is_copy_constructible_v<ValueType>` is true. Otherwise the program is ill-formed.

-5- *Returns:* For the first form, `*any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand);` For the and second and third forms, `*any_cast<remove_reference_t<ValueType>>(&operand);` For the third form, `std::forward<ValueType>(*any_cast<remove_reference_t<ValueType>>(&operand));`

[...]

[2016-10-05, Tomasz and Casey reopen and improve the wording]

The constraints placed on the non-pointer `any_cast` overloads are neither necessary nor sufficient to guarantee that the specified effects are well-formed. The current PR for LWG [2769](#) also makes it possible to retrieve a dangling lvalue reference to a temporary any with e.g. `any_cast<int&>(any{42})`, which should be forbidden.

[2016-10-16, Eric made some corrections to the wording]

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Modify 20.8.4 [any.nonmembers] as indicated:

```
template<class ValueType>
    ValueType any_cast(const any& operand);
template<class ValueType>
    ValueType any_cast(any& operand);
template<class ValueType>
    ValueType any_cast(any&& operand);
```

-4- *Requires:* `is_reference_v<ValueType>` is true OR `is_copy_constructible_v<ValueType>` is true. For the first overload, `is_constructible_v<ValueType, const remove_cv_t<remove_reference_t<ValueType>>&>` is true. For the second overload, `is_constructible_v<ValueType, remove_cv_t<remove_reference_t<ValueType>>&>` is true. For the third overload, `is_constructible_v<ValueType, remove_cv_t<remove_reference_t<ValueType>>>` is true. Otherwise the program is ill-formed.

-5- *Returns:* For the first form, `*any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand);` For the second and third forms, `*any_cast<remove_reference_t<ValueType>>(&operand);` For the first and second overload, `static cast<ValueType>(*any_cast<remove_cv_t<remove_reference_t<ValueType>>>(&operand));` For the third overload, `static cast<ValueType>(std::move(*any_cast<remove_cv_t<remove_reference_t<ValueType>>>(&operand)));`

[...]

[2016-11-10, LWG asks for simplification of the wording]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.8.4 [any.nonmembers] as indicated:

```
template<class ValueType>
    ValueType any_cast(const any& operand);
template<class ValueType>
    ValueType any_cast(any& operand);
template<class ValueType>
    ValueType any_cast(any&& operand);
```

-?- Let U be the type `remove_cv_t<remove_reference_t<ValueType>>`.

-4- *Requires:* `is_reference_v<ValueType> is true OR is_copy_constructible_v<ValueType> is true`. For the first overload, `is_constructible_v<ValueType, const U&> is true`. For the second overload, `is_constructible_v<ValueType, U&> is true`. For the third overload, `is_constructible_v<ValueType, U> is true`. Otherwise the program is ill-formed.

-5- *Returns:* For the first form, `*any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand)`. For the second and third forms, `*any_cast<remove_reference_t<ValueType>>(&operand)`. For the first and second overload, `static cast<ValueType>(*any_cast<U>(&operand))`. For the third overload, `static cast<ValueType>(std::move(*any_cast<U>(&operand)))`.

[...]

2781. Contradictory requirements for `std::function` and `std::reference_wrapper`

Section: 20.14.13.2.1 [func.wrap.func.con] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-10-13 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [func.wrap.func.con].

View all other [issues](#) in [func.wrap.func.con].

View all issues with [Ready](#) status.

Discussion:

`template<class F> function(F f)` says that the effects are "`*this` targets a copy of `f`" which seems pretty clear that if `F` is `reference_wrapper<CallableType>` then the target is a `reference_wrapper<CallableType>`.

But the function copy and move constructors say "shall not throw exceptions if `f`'s target is a callable object passed via `reference_wrapper` or a function pointer." From the requirement above it's impossible for the target to be "a callable object passed via `reference_wrapper`" because if the function was constructed with such a type then the target is the `reference_wrapper` not the callable object it wraps.

This matters because it affects the result of `function::target_type()`, and we have implementation divergence. VC++ and libc++ store the `reference_wrapper` as the target, but libstdc++ and Boost.Function (both written by Doug Gregor) unwrap it, so the following fails:

```
#include <functional>
#include <cassert>

int main()
{
    auto f = []{};
    std::function<void()> fn(std::ref(f));
    assert(fn.target<std::reference_wrapper<decltype(f)>>() != nullptr);
}
```

If `std::function` is intended to deviate from `boost::function` this way then the *Throws* element for the copy and move constructors is misleading, and should be clarified.

Sat AM: Priority 0; move to Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.14.13.2.1 [func.wrap.func.con] p4 and p6 the same way, as shown:

```
function(const function& f);
```

-3- *Postconditions*: `!*this` if `!f`; otherwise, `*this` targets a copy of `f.target()`.

-4- *Throws*: shall not throw exceptions if `f`'s target is a ~~callable object passed via~~ specialization of `reference_wrapper` or a function pointer. Otherwise, may throw `bad_alloc` or any exception thrown by the copy constructor of the stored callable object. [Note: Implementations are encouraged to avoid the use of dynamically allocated memory for small callable objects, for example, where `f`'s target is an object holding only a pointer or reference to an object and a member function pointer. — end note]

```
function(function&& f);
```

-5- *Effects*: If `!f`, `*this` has no target; otherwise, `move` constructs the target of `f` into the target of `*this`, leaving `f` in a valid state with an unspecified value.

-6- *Throws*: shall not throw exceptions if `f`'s target is a ~~callable object passed via~~ specialization of `reference_wrapper` or a function pointer. Otherwise, may throw `bad_alloc` or any exception thrown by the copy or move constructor of the stored callable object. [Note: Implementations are encouraged to avoid the use of dynamically allocated memory for small callable objects, for example, where `f`'s target is an object holding only a pointer or reference to an object and a member function pointer. — end note]

[2782](#). `scoped_allocator_adaptor` constructors must be constrained

Section: 20.13.3 [allocator.adaptor.cnstr] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-10-14 **Last modified:** 2017-02-03

Priority: 0

View all issues with [Ready](#) status.

Discussion:

The templated constructors of `scoped_allocator_adaptor` need to be constrained, otherwise `uses_allocator` construction gives the wrong answer and causes errors for code that should compile.

Consider two incompatible allocator types:

```
template<class T> struct Alloc1 { ... };
template<class T> struct Alloc2 { ... };
static_assert(!is_convertible_v<Alloc1<int>, Alloc2<int>>);
```

The unconstrained constructors give this bogus answer:

```
template<class T> using scoped = scoped_allocator_adaptor<T>;
static_assert(is_convertible_v<scoped<Alloc1<int>>, scoped<Alloc2<int>>>);
```

This causes `uses_allocator` to give the wrong answer for any specialization involving incompatible `scoped_allocator_adaptors`, which makes `scoped_allocator_adaptor::construct()` take an ill-formed branch e.g.

```
struct X
{
    using allocator_type = scoped<Alloc2<int>>;
    X(const allocator_type&);
    X();
};
scoped<Alloc1<int>>{}.construct((X*)0);
```

This fails to compile, because `uses_allocator<X, scoped_allocator_adaptor<Alloc2<int>>>` is true, so the allocator is passed to the `X` constructor, but the conversion fails. The error is outside the immediate context, and so is a hard error.

[2016-11-12, Issaquah]

Sat AM: Priority 0; move to Ready

Billy to open another issue about the confusion with the ctor

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.13.3 [allocator.adaptor.cnstr] by converting "Requires" elements to "Remarks: shall not participate ..." constraints as shown:

```
template <class OuterA2>
    scoped_allocator_adaptor(OuterA2&& outerAlloc,
                             const InnerAllocs&... innerAllocs) noexcept;
```

~~-2- Requires: OuterAlloc shall be constructible from OuterA2.~~

-3- *Effects*: Initializes the OuterAlloc base class with `std::forward<OuterA2>(outerAlloc)` and inner with `innerAllocs...` (hence recursively initializing each allocator within the adaptor with the corresponding allocator from the argument list).

~~-?- Remarks: This constructor shall not participate in overload resolution unless `is_constructible_v<OuterAlloc, OuterA2>` is true.~~

[...]

```
template <class OuterA2>
    scoped_allocator_adaptor(const scoped_allocator_adaptor<OuterA2,
                             InnerAllocs...>& other) noexcept;
```

~~-6- Requires: OuterAlloc shall be constructible from OuterA2.~~

-7- *Effects*: Initializes each allocator within the adaptor with the corresponding allocator from `other`.

~~-?- Remarks: This constructor shall not participate in overload resolution unless `is_constructible_v<OuterAlloc, const OuterA2&>` is true.~~

```
template <class OuterA2>
    scoped_allocator_adaptor(scoped_allocator_adaptor<OuterA2,
                             InnerAllocs...>&& other) noexcept;
```

~~-8- Requires: OuterAlloc shall be constructible from OuterA2.~~

-9- *Effects*: Initializes each allocator within the adaptor with the corresponding allocator rvalue from `other`.

~~-?- Remarks: This constructor shall not participate in overload resolution unless `is_constructible_v<OuterAlloc, OuterA2>` is true.~~

[2784](#). Resolution to LWG 2484 is missing "otherwise, no effects" and is hard to parse

Section: 18.8.7 [except.nested] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-10-15 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [except.nested].

View all other [issues](#) in [except.nested].

View all issues with [Ready](#) status.

Discussion:

The discussion notes for LWG [2484](#) point out there should be an "else, no effects" at the end, which didn't make it into the resolution. Without this it's unclear whether it should do nothing, or be ill-formed, or undefined.

Additionally, the precise effects of the *Effects* are hard to determine, because the conditions on the static type and dynamic type are intermingled, but must actually be checked separately (the static checks must be done statically and the dynamic checks

must be done dynamically!) Furthermore, the obvious way to know if "the dynamic type of `e` is `nested_exception` or is publicly and unambiguously derived from `nested_exception`" is to use `dynamic_cast`, so we have to use `dynamic_cast` to find out whether to perform the `dynamic_cast` expression specified in the *Effects*. It would make more sense to specify it in terms of a `dynamic_cast` to a pointer type, and only call `rethrow_nested()` if the result is not null.

The entire spec can be expressed in C++17 as:

```
if constexpr(is_polymorphic_v<E> && (!is_base_of_v<nested_exception, E> || is_convertible_v<E*, nested_exception*>))
    if (auto p = dynamic_cast<const nested_exception*>(addressof(e)))
        p->rethrow_nested();
```

This uses traits to perform checks on the static type, then uses `dynamic_cast` to perform the checks on the dynamic type. I think the spec would be clearer if it had the same structure.

[2016-11-12, Issaquah]

Sat AM: Priority 0; move to Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 18.8.7 [except.nested] p9:

```
template <class E> void rethrow_if_nested(const E& e);
```

-9- *Effects*: If `E` is not a polymorphic class type, or if `nested_exception` is an inaccessible or ambiguous base class of `E`, there is no effect. Otherwise, if the static type or the dynamic type of `e` is `nested_exception` or is publicly and unambiguously derived from `nested_exception`, callsperforms:

```
dynamic_cast<const nested_exception*>(e)->rethrow_nested();
if (auto p = dynamic_cast<const nested_exception*>(addressof(e)))
    p->rethrow_nested();
```

2785. quoted should work with basic_string_view

Section: 27.7.8 [quoted.manip] **Status:** [Ready](#) **Submitter:** Marshall Clow **Opened:** 2016-10-27 **Last modified:** 2017-02-03

Priority: 0

View all other [issues](#) in [quoted.manip].

View all issues with [Ready](#) status.

Discussion:

Quoted output for strings was added for C++14. But when we merged `string_view` from the Library Fundamentals TS, we did not add support for quoted output of `basic_string_view`

[2016-11-12, Issaquah]

Sat AM: Priority 0; move to Ready

13 -> 13 in paragraph modification; and T14 -> T15

Proposed resolution:

This wording is relative to N4606.

Add to the end of the `<iomanip>` synopsis in [iostream.format.overview]

```
template <class charT, class traits>
    T15 quoted(basic_string_view<charT, traits> s,
               charT delim = charT('"'), charT escape = charT('\\'));
```

Add to [quoted.manip] at the end of p2:

```
template <class charT, class traits>
    unspecified quoted(basic_string_view<charT, traits> s,
                      charT delim = charT('"'), charT escape = charT('\\'));
```


Modify [quoted.manip]/3 as follows:

Returns: An object of unspecified type such that if out is an instance of `basic_ostream` with member type `char_type` the same as `charT` and with member type `traits_type` which in the second and third forms is the same as `traits`, then the expression `out << quoted(s, delim, escape)` behaves as a formatted output function (27.7.3.6.1) of out. This forms a character sequence seq, initially consisting of the following elements:

2786. Annex C should mention `shared_ptr` changes for array support

Section: C.4.9 [diff.cpp14.utilities] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-10-18 **Last modified:** 2017-02-03

Priority: 0

View all issues with [Ready](#) status.

Discussion:

This is valid in C++11 and C++14:

```
shared_ptr<int> s{unique_ptr<int[]>{new int[1]}};
```

The `shared_ptr` copies the `default_delete<int[]>` deleter from the `unique_ptr`, which does the right thing on destruction.

In C++17 it won't compile, because `!is_convertible_v<int(*)[], int*>`.

The solution is to use `shared_ptr<int[]>`, which doesn't work well in C++14, so there's no transition path. This should be called out in Annex C.

[2016-11-12, Issaquah]

Sat AM: Priority 0; move to Ready

Proposed resolution:

This wording is relative to N4606.

1. Add to C.4.9 [diff.cpp14.utilities]:

20.11.2.2 [util.smartptr.shared]

Change: Different constraint on conversions from `unique_ptr`.

Rationale: Adding array support to `shared_ptr`, via the syntax `shared_ptr<T[]>` and `shared_ptr<T[N]>`.

Effect on original feature: Valid code may fail to compile or change meaning in this International Standard. For example:

```
#include <memory>
std::unique_ptr<int[]> arr(new int[1]);
std::shared_ptr<int> ptr(std::move(arr)); // error: int(*)[] is not compatible with int*
```

2787. `file_status.cons` doesn't match class definition

Section: 27.10.11 [class.file_status], 27.10.11.1 [file_status.cons] **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2016-10-21 **Last modified:** 2017-02-03

Priority: 0

View all issues with [Ready](#) status.

Discussion:

27.10.11 [class.file_status] depicts:

```
explicit file_status(file_type ft = file_type::none, perms prms = perms::unknown) noexcept;
```

while 27.10.11.1 [file_status.cons] describes two constructors:

```
explicit file_status() noexcept;
explicit file_status(file_type ft, perms prms = perms::unknown) noexcept;
```

It's also not clear why the default constructor needs to be explicit. Unlike tag types, there doesn't seem to be a compelling reason to disallow constructing a `file_status` without naming the type.

[2016-11-12, Issaquah]

Sat AM: Priority 0; move to Ready

Proposed resolution:

This wording is relative to N4606.

1. Edit 27.10.11 [class.file_status] as indicated:

```
class file_status {
public:
    // 27.10.11.1, constructors and destructor:
    file_status() noexcept : file_status(file_type::none) {}
    explicit file_status(file_type ft = file_type::none,
                        perms prms = perms::unknown) noexcept;

    [...]
};
```

2. Edit 27.10.11.1 [file_status.cons] as indicated:

```
explicit file_status() noexcept;

-1- Postconditions: type() == file_type::none and permissions() == perms::unknown;
```

2789. Equivalence of contained objects

Section: 20.8.3 [any.class] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-11-09 **Last modified:** 2017-02-03

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses US 29

What does it mean for (the contained) objects to be "equivalent"?

Suggested resolution:

Add definition (note that using `operator==( )` involves complicated questions of overload resolution).

[2016-11-08, Jonathan comments and suggests wording]

We can rephrase the copy constructor in terms of equivalence to construction from the contained object. We need to use in-place construction to avoid recursion in the case where the contained object is itself an any.

For the move constructor we don't simply want to construct from the contained object, because when the contained object is stored in dynamic memory we don't actually construct anything, we just transfer ownership of a pointer.

[Issues Telecon 16-Dec-2016]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change [any.cons] p2:

```
any(const any& other);
```

```
Effects: Constructs an object of type any with an equivalent state as other. If other.has_value() is false,
constructs an object that has no value. Otherwise, equivalent to any(in_place<T>, any_cast<const T&>
(other)) where T is the type of the contained object.
```

2. Change [any.cons] p4:

```
any(any&& other);
```

Effects: Constructs an object of type `any` with a state equivalent to the original state of `other`. If `other.has_value()` is false, constructs an object that has no value. Otherwise, constructs an object of type `any` that contains either the contained object of `other`, or contains an object of the same type constructed from the contained object of `other` considering that contained object as an rvalue.

[2794](#). Missing requirements for allocator pointers

Section: 23.2.1 [container.requirements.general] **Status:** [Tentatively Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2016-11-09 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [container.requirements.general].

View all other [issues](#) in [container.requirements.general].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses US 146

An allocator-aware contiguous container must require an allocator whose pointer type is a contiguous iterator. Otherwise, functions like `data` for `basic_string` and `vector` do not work correctly, along with many other expectations of the contiguous guarantee.

Suggested resolution:

Add a second sentence to 23.2.1 [container.requirements.general] p13:

An allocator-aware contiguous container requires `allocator_traits<Allocator>::pointer` is a contiguous iterator.

[2016-11-12, Issaquah]

Sat PM: Move to "Tentatively Ready"

Proposed resolution:

This wording is relative to N4606.

1. In 17.5.3.5 [allocator.requirements]/5, edit as follows:

-5- An allocator type `X` shall satisfy the requirements of `CopyConstructible` (17.6.3.1). The `X::pointer`, `X::const_pointer`, `X::void_pointer`, and `X::const_void_pointer` types shall satisfy the requirements of `NullablePointer` (17.6.3.3). No constructor, comparison operator, copy operation, move operation, or swap operation on these pointer types shall exit via an exception. `X::pointer` and `X::const_pointer` shall also satisfy the requirements for a random access iterator (24.2 [iterator.requirements] 24.2.7 [random.access.iterators]) and of a contiguous iterator (24.2.1 [iterator.requirements.general]).

[2795](#). §[global.functions] provides incorrect example of ADL use

Section: 17.5.5.4 [global.functions] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-11-09 **Last modified:** 2017-02-03

Priority: Not Prioritized

View all other [issues](#) in [global.functions].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses GB 39

The example is supposed to highlight the 'otherwise specified' aspect of invoking ADL, yet there is no such specification. It is unlikely that we intend to explicitly qualify calls to operator functions, so they probably should be exempted from this restriction.

Suggested resolution:

Fix example (and referenced clause) to specify use of ADL, or exempt operators from this clause, and find a better example, probably using `swap`.

[2016-11-09, Jonathan comments and provides wording]

The current wording was added by DR [225](#).

[2016-11-10, Tim Song comments]

The "non-operator" seems to have been added at the wrong spot. The problem at issue is permission to call operator functions found via ADL, not permission for operator functions in the standard library to ADL all over the place. The problem is not unique to operator functions in the standard library — a significant portion of `<algorithm>` and `<numeric>` uses some operator (`==`, `<`, `+`, `*`, etc.) that may be picked up via ADL.

There is also an existing problem in that the example makes no sense anyway: the constraint in this paragraph only applies to non-members, yet `ostream_iterator::operator=` is a member function.

[2016-11-10, Tim Song and Jonathan agree on new wording]

The new wording still doesn't quite get it right:

"calls to non-operator, non-member functions in the standard library do not use functions from another namespace which are found through argument-dependent name lookup" can be interpreted as saying that if a user writes `std::equal(a, b, c, d)`, that call will not use ADL'd `operator==` because `std::equal(a, b, c, d)` is a "call" to a "non-operator, non-member function in the standard library".

The key point here is that "in the standard library" should be modifying "calls", not "function".

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Change 17.5.5.4 [global.functions] p4:

Unless otherwise specified, **non-operator**, non-member functions in the standard library shall not use functions from another namespace which are found through argument-dependent name lookup (3.4.2). [Note: The phrase "unless otherwise specified" **applies to cases such as the swappable with requirements (17.5.3.2 [swappable.requirements]). The exception for overloaded operators allows**~~is intended to allow~~ argument-dependent lookup in cases like that of `ostream_iterator::operator=` (24.6.2.2):

Effects:

```
*out_stream << value;
if (delim != 0)
    *out_stream << delim ;
return *this;
```

— end note]

[Issues Telecon 16-Dec-2016]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change 17.5.5.4 [global.functions] p4:

Unless otherwise specified, **calls made by functions in the standard library to non-operator**, non-member functions ~~in the standard library shall~~**do** not use functions from another namespace which are found through

argument-dependent name lookup (3.4.2). [Note: The phrase "unless otherwise specified" [applies to cases such as the swappable with requirements \(17.5.3.2 \[swappable.requirements\]\). The exception for overloaded operators allows is intended to allow](#) argument-dependent lookup in cases like that of `ostream_iterator::operator=` (24.6.2.2):

Effects:

```
*out_stream << value;
if (delim != 0)
    *out_stream << delim ;
return *this;
```

— *end note*

[2804](#). Unconditional constexpr default constructor for `istream_iterator`

Section: 24.6.1.1 [istream.iterator.cons] **Status:** [Tentatively Ready](#) **Submitter:** Marshall Clow **Opened:** 2016-11-09 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [istream.iterator.cons].

View all other [issues](#) in [istream.iterator.cons].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses US 152

see below for the default constructor should simply be spelled `constexpr`. The current declaration looks like a member function, not a constructor, and the `constexpr` keyword implicitly does not apply unless the instantiation could make it so, under the guarantees already present in the Effects clause.

Proposed change:

Replace *see below* with `constexpr` in the declaration of the default constructor for `istream_iterator` in the class definition, and function specification.

[Issues Telecon 16-Dec-2016]

Jonathan provides wording, Move to Tentatively Ready

Proposed resolution:

In the class synopsis in 24.6.1 [istream.iterator] change the default constructor:

```
see belowconstexpr istream_iterator();
istream_iterator(istream_type& s);
istream_iterator(const istream_iterator& x) = default;
~istream_iterator() = default;
```

Change [istream.iterator.cons] before paragraph 1:

```
see belowconstexpr istream_iterator();

-1- Effects: ...
```

[2812](#). Range access is available with `<string_view>`

Section: 24.7 [iterator.range], 21.4.1 [string.view.synop] **Status:** [Tentatively Ready](#) **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2016-10-29 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [iterator.range].

View all other [issues](#) in [iterator.range].

View all issues with [Tentatively Ready](#) status.

Discussion:

21.4.1 [string.view.synop]/1 states

The function templates defined in 20.2.2 and 24.7 are available when `<string_view>` is included.

24.7 [iterator.range], in p1 is missing `<string_view>`.

[Issues Telecon 16-Dec-2016]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Edit 24.7 [iterator.range] p1 as indicated:

-1- In addition to being available via inclusion of the `<iterator>` header, the function templates in 24.7 are available when any of the following headers are included: `<array>`, `<deque>`, `<forward_list>`, `<list>`, `<map>`, `<regex>`, `<set>`, `<string>`, `<string_view>`, `<unordered_map>`, `<unordered_set>`, and `<vector>`.

[2824](#). `list::sort` should say that the order of elements is unspecified if an exception is thrown

Section: 23.3.10.5 [list.ops] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-11-16 **Last modified:** 2017-02-05

Priority: 0

View all other [issues](#) in [list.ops].

View all issues with [Tentatively Ready](#) status.

Discussion:

Sometime between N1638 and N1804 the sentence "If an exception is thrown the order of the elements in the list is indeterminate." in the specification of `list::sort` went missing. This suspiciously coincided with the editorial change that "consolidated definitions of "Stable" in the library clauses" (N1805).

`forward_list::sort` says that "If an exception is thrown the order of the elements in `*this` is unspecified"; `list::sort` should do the same.

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to N4606.

1. Edit 23.3.10.5 [list.ops] p29 as indicated:

```
void sort();
template <class Compare> void sort(Compare comp);
```

-28- *Requires:* `operator<` (for the first version) or `comp` (for the second version) shall define a strict weak ordering (25.7 [alg.sorting]).

-29- *Effects:* Sorts the list according to the `operator<` or a `Compare` function object. If an exception is thrown, the order of the elements in `*this` is unspecified. Does not affect the validity of iterators and references.

[...]

[2826](#). `string_view` iterators use old wording

Section: 21.4.2.2 [string.view.iterators] **Status:** [Tentatively Ready](#) **Submitter:** Alisdair Meredith **Opened:** 2016-11-17 **Last**

modified: 2017-02-03

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The wording for `basic_string_view` was written before the definition of a contiguous iterator was added to C++17 to avoid repeating redundant wording.

Suggested modification of 21.4.2.2 [string.view.iterators] p1 (stealing words from `valarray` begin/end):

-1- ~~A constant random-access iterator type such that, for a `const_iterator` `it`, if `&*(it + N)` is valid, then `&*(it + N) == (&*it) + N`~~ A type that meets the requirements of a constant random access iterator (24.2.7) and of a contiguous iterator (24.2.1) whose value type is the template parameter `charT`.

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to N4606.

1. Modify 21.4.2.2 [string.view.iterators] as indicated:

```
using const_iterator = implementation-defined;
```

-1- ~~A constant random-access iterator type such that, for a `const_iterator` `it`, if `&*(it + N)` is valid, then `&*(it + N) == (&*it) + N`~~ type that meets the requirements of a constant random access iterator (24.2.7 [random.access.iterators]) and of a contiguous iterator (24.2.1 [iterator.requirements.general]) whose value type is the template parameter `charT`.

-2- For a `basic_string_view` `str`, any operation that invalidates a pointer in the range `[str.data(), str.data() + str.size())` invalidates pointers, iterators, and references returned from `str`'s methods.

-3- All requirements on container iterators (23.2 [container.requirements]) apply to `basic_string_view::const_iterator` as well.

2834. Resolution LWG 2223 is missing wording about end iterators

Section: 21.3.2.4 [string.capacity], 23.3.8.3 [deque.capacity], 23.3.11.3 [vector.capacity] **Status:** [Tentatively Ready](#) **Submitter:** Thomas Koepp **Opened:** 2016-11-29 **Last modified:** 2017-02-03

Priority: 0

View all other [issues](#) in [string.capacity].

View all issues with [Tentatively Ready](#) status.

Discussion:

The resolution of LWG 2223 added the wording "Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence." to a number of `shrink_to_fit` operations.

This seems to be missing any mention of end iterators. Surely end iterators are invalidated, too?

Suggested resolution:

For `string`, `deque`, and `vector` each, append as follows:

Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator.

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to [N4618](#).

1. Edit 21.3.2.4 [string.capacity] as indicated:

```
void shrink_to_fit();
```

[...]

-15- *Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator. If no reallocation happens, they remain valid.

2. Edit 23.3.8.3 [deque.capacity] as indicated:

```
void shrink_to_fit();
```

[...]

-8- *Remarks:* shrink_to_fit invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator.

3. Edit 23.3.11.3 [vector.capacity] as indicated:

```
void shrink_to_fit();
```

[...]

-10- *Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator. If no reallocation happens, they remain valid.

[2835](#). LWG 2536 seems to misspecify <tgmath.h>

Section: 26.9.6 [ctgmath.syn], D.4 [depr.c.headers] **Status:** [Tentatively Ready](#) **Submitter:** Thomas Koeppel **Opened:** 2016-11-29
Last modified: 2017-02-03

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The resolution of LWG [2536](#) touches on the specification of C headers (of the form `foo.h`), but while it fixes the specification of `complex.h`, it fails to address the specification of `tgmath.h`.

Just like `complex.h`, `tgmath.h` is **not** defined by C. Rather, *our* `tgmath.h` behaves like `<ctgmath>`, which is specified in [ctgmath.syn].

Suggested resolution:

Amend [depr.c.headers] p3 as follows:

3. The header `<complex.h>` behaves as if it simply includes the header `<ccomplex>`. The header `<tgmath.h>` behaves as if it simply includes the header `<ctgmath>`.

4. Every other C header [...]

See also LWG [2828](#).

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to [N4618](#).

1. Edit D.4 [depr.c.headers] as indicated:

[...]

-2- The use of any of the C++ headers [...] is deprecated.

-3- The header `<complex.h>` behaves as if it simply includes the header `<complex>`. The header `<tgmath.h>` behaves as if it simply includes the header `<tgmath>`.

-4- Every other C header, [...]

2837. gcd and lcm should support a wider range of input values

Section: 26.8.13 [numeric.ops.gcd], 26.8.14 [numeric.ops.lcm] **Status:** [Tentatively Ready](#) **Submitter:** Marshall Clow **Opened:** 2016-12-16 **Last modified:** 2017-02-03

Priority: 0

View all other [issues](#) in [numeric.ops.gcd].

View all issues with [Tentatively Ready](#) status.

Discussion:

This is a duplicate of [2792](#), which addressed LFTS 2.

By the current definition, `gcd((int64_t)1234, (int32_t)-2147483648)` is ill-formed (because 2147483648 is not representable as a value of `int32_t`.) We want to change this case to be well-formed. As long as both `|m|` and `|n|` are representable as values of the common type, absolute values can be calculate d without causing unspecified behavior, by converting `m` and `n` to the common type before taking the negation.

Suggested resolution:

~~`|m| shall be representable as a value of type M and |n| shall be representable as a value of type N`~~ `|m| and |n| shall be representable as a value of common type t<M, N>.`

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to [N4604](#).

1. Edit 26.8.13 [numeric.ops.gcd] as indicated:

```
template<class M, class N>
constexpr common_type_t<M, N> gcd(M m, N n);
```

-2- *Requires:* ~~`|m| shall be representable as a value of type M and |n| shall be representable as a value of type N`~~ `|m| and |n| shall be representable as a value of common type t<M, N>.` [Note: These requirements ensure, for example, that `gcd(m, m) = |m|` is representable as a value of type `M`. — end note]

2. Edit 26.8.14 [numeric.ops.lcm] as indicated:

```
template<class M, class N>
constexpr common_type_t<M, N> lcm(M m, N n);
```

-2- *Requires:* ~~`|m| shall be representable as a value of type M and |n| shall be representable as a value of type N`~~ `|m| and |n| shall be representable as a value of common type t<M, N>.` The least common multiple of `|m|` and `|n|` shall be representable as a value of type `common_type_t<M, N>`.

2838. is_literal_type specification needs a little cleanup

Section: D.11 [depr.meta.types] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-12-09 **Last modified:** 2017-02-03

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

D.11 [depr.meta.types]/3 currently reads:

Effects: `is_literal_type` has a base-characteristic of `true_type` if τ is a literal type ([basic.types]), and a base-characteristic of `false_type` otherwise.

First, this doesn't seem like an *Effects* clause. Second, this wording fails to say that `is_literal_type` is an `UnaryTypeTrait`, and misspells `BaseCharacteristic` — which is only defined for `UnaryTypeTraits` and `BinaryTypeTraits`. Third, moving this to Annex D means that the general prohibition against specializing type traits in [meta.type.synop]/1 no longer applies, which is presumably unintended.

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to [N4618](#).

1. Edit D.11 [depr.meta.types] as indicated:

The header `<type_traits>` has the following addition:

```
namespace std {
    template <class T> struct is_literal_type;

    template <class T> constexpr bool is_literal_type_v = is_literal_type<T>::value;
}
```

-2- *Requires:* `remove_all_extents_t<T>` shall be a complete type or (possibly cv-qualified) void.

-3- *Effects:* `is_literal_type` has a base-characteristic of `true_type` if τ is a literal type (3.9), and a base-characteristic of `false_type` otherwise. `is_literal_type<T>` is a `UnaryTypeTrait` (20.15.1 [meta.rgmts]) with a `BaseCharacteristic` of `true_type` if τ is a literal type (3.9 [basic.types]), and `false_type` otherwise.

-?- The behavior of a program that adds specializations for `is_literal_type` or `is_literal_type_v` is undefined.

[2842](#). `in_place_t` check for `optional(U&&)` should decay `U`

Section: 20.6.3.1 [optional.ctor] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-12-13 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [optional.ctor].

View all other [issues](#) in [optional.ctor].

View all issues with [Tentatively Ready](#) status.

Discussion:

As `in_place_t` is a normal tag type again, we need to decay `U` before doing the `is_same_v` check.

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to [N4618](#).

1. Edit 20.6.3.1 [optional.ctor] as indicated:

```
template <class U = T>
    EXPLICIT constexpr optional(U&& v);
```

[...]

-22- *Remarks:* If τ 's selected constructor is a `constexpr` constructor, this constructor shall be a `constexpr` constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>` is true, `is_same_v<decay_t<U>, in_place_t>` is false, and `is_same_v<optional<T>, decay_t<U>>` is false.

2850. `std::function` move constructor does unnecessary work

Section: 20.14.13.2.1 [func.wrap.func.con] **Status:** [Tentatively Ready](#) **Submitter:** Geoffrey Romer **Opened:** 2016-12-15 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [func.wrap.func.con].

View all other [issues](#) in [func.wrap.func.con].

View all issues with [Tentatively Ready](#) status.

Discussion:

Consider [func.wrap.func.con]/p5:

```
function(function&& f);
```

Effects: If !f, *this has no target; otherwise, move constructs the target of f into the target of *this, leaving f in a valid state with an unspecified value.

By my reading, this wording requires the move constructor of `std::function` to construct an entirely new target object. This is silly: in cases where the target is held in separately allocated memory (i.e. where the target doesn't fit in `std::function`'s internal buffer, if any), `std::function`'s move constructor can be implemented by simply transferring ownership of the existing target object (which is a simple pointer assignment), so this requirement forces an unnecessary constructor invocation and probably an unnecessary allocation (the latter can be avoided with something like double-buffering, but ew). Fixing this would technically be a visible change, but I have a hard time imagining reasonable code that would be broken by it, especially since both `libstdc++` and `libc++` already do the sensible thing, constructing a new target only if the target is held in an internal buffer, and otherwise assigning pointers.

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to [N4618](#).

1. Edit 20.14.13.2.1 [func.wrap.func.con]/5 as indicated:

Drafting note: The "equivalent to ... before the construction" wording is based on the wording for `MoveConstructible`.

```
function(function&& f);
```

-5- ~~*Effects*~~*Postconditions:* If !f, *this has no target; otherwise, ~~move constructs the target of f into the target of *this, leaving f~~*the target of *this is equivalent to the target of f before the construction, and f* ~~is~~ in a valid state with an unspecified value.

2853. Possible inconsistency in specification of `erase` in [vector.modifiers]

Section: 23.3.11.5 [vector.modifiers] **Status:** [Tentatively Ready](#) **Submitter:** Gerard Stone **Opened:** 2017-01-16 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [vector.modifiers].

View all other [issues](#) in [vector.modifiers].

View all issues with [Tentatively Ready](#) status.

Discussion:

In Table 87 (Sequence container requirements) `erase(q)` and `erase(q1, q2)` functions have the following requirements:

For vector and deque, T shall be `MoveAssignable`.

On the other hand, section [vector.modifiers] has the following specification for erase functions (emphasis mine):

Throws: Nothing unless an exception is thrown by the **copy constructor**, **move constructor**, assignment operator, or move assignment operator of T .

Note that Table 87 requires T to be only `MoveAssignable`, it says nothing about T being copy- or move-constructible. It also says nothing about T being `CopyInsertable` and `MoveInsertable`, so why is this even there? The only reason might be so that vector could shrink, but in this case T should be required to be `MoveInsertable` OR `CopyInsertable` into vector.

On the other hand, we expect that vector will neither allocate, nor deallocate any memory during this operation, because in *Effects* it is specified that iterators/references shall be invalidated at or after the point of the erase.

So to avoid any confusion, the proposed resolution is to remove mentions of T 's copy/move constructors from [vector.modifiers] paragraph 5.

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to [N4618](#).

1. Edit 23.3.11.5 [vector.modifiers] p5 as indicated:

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void pop_back();
```

-3- *Effects:* Invalidates iterators and references at or after the point of the erase.

-4- *Complexity:* The destructor of T is called the number of times equal to the number of the elements erased, but the assignment operator of T is called the number of times equal to the number of elements in the vector after the erased elements.

-5- *Throws:* Nothing unless an exception is thrown by the ~~copy constructor, move constructor,~~ assignment operator, or move assignment operator of T .

[2855](#). `std::throw_with_nested("string_literal")`

Section: 18.8.7 [except.nested] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2017-01-17 **Last modified:** 2017-02-03

Priority: 0

View other [active issues](#) in [except.nested].

View all other [issues](#) in [except.nested].

View all issues with [Tentatively Ready](#) status.

Discussion:

[except.nested] says:

```
template <class T> [[noreturn]] void throw_with_nested(T&& t);
```

Let U be `remove_reference_t<T>`.

Requires: U shall be `CopyConstructible`.

This forbids `std::throw_with_nested("string_literal")` because T gets deduced as `const char(&)[15]` and so U is `const char[15]` which is not `CopyConstructible`.

A throw expression decays an array argument to a pointer (5.17 [expr.throw] p2) and so works fine with string literals. GCC's `throw_with_nested` also worked fine until I added a `static_assert` to enforce the `CopyConstructible` requirement.

The same problem exists when throwing a function type, which should also decay:

```
#include <exception>

void f() { }

int main() {
    std::throw_with_nested(f);
}
```

(Note: LWG [1370](#) added the `remove_reference`, which was a step in the right direction but not far enough.)

[2017-01-27 Telecon]

Priority 0

Proposed resolution:

This wording is relative to [N4618](#).

1. Edit 18.8.7 [except.nested] as indicated:

```
template <class T> [[noreturn]] void throw_with_nested(T&& t);
```

-6- Let U be ~~remove_reference~~decay_t<T>.

-7- *Requires:* U shall be CopyConstructible.